RFC — Triple P Bank: banco digital com investimentos simulados, sem criar nem perder um centavo

Time	Lucas Heluy · <nome 2> · <nome 3>
Data	07/10/2026
Versão	1

Contextualização

Entendendo o problema

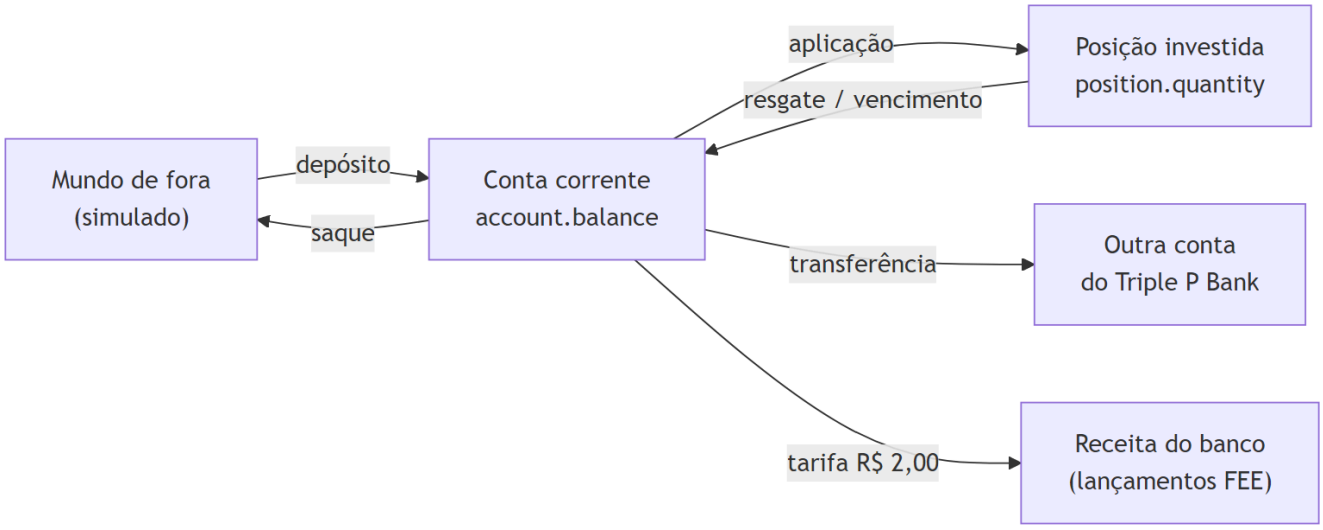
O Triple P Bank é a API de um banco digital. Quem a usa é um aplicativo ou integrador agindo em nome de clientes pessoa física, e a operação interna do banco. Para o cliente, ela cadastra, abre a conta corrente, deposita, saca, transfere com tarifa e mostra o extrato. No módulo de investimentos, ela calcula o perfil do investidor, sugere uma carteira, aplica e resgata em renda fixa e no fundo fictício FGV Quant Capital e mostra posição e rentabilidade. Para a operação, ela vira o dia, com os preços buscados lá fora, e mede a receita do banco.

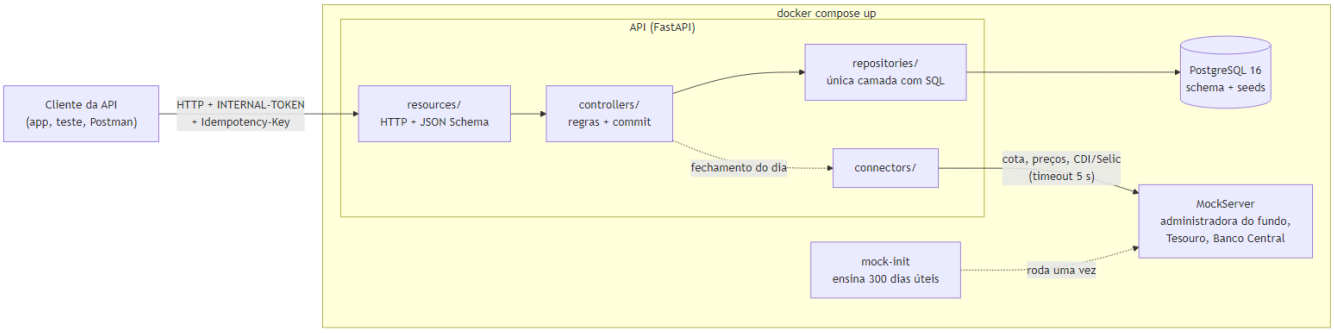
A garantia é que nenhum centavo é criado, duplicado ou some. O saldo nunca fica negativo. O extrato sempre reconstrói o saldo. Um pedido reenviado depois de um timeout não move o dinheiro duas vezes. Uma falha no meio não deixa débito sem crédito ou sem posição. Nada do que foi registrado é apagado. Um cliente nunca enxerga o recurso de outro. Se isso falha, o banco paga em dobro, o cliente perde dinheiro ou o extrato deixa de servir como prova.

Fora do escopo: autenticação do cliente final, Pix/TED reais, pessoa jurídica, crédito, ações, cripto e preço em tempo real, imposto de renda e IOF, feriados e front-end.

Explicando a solução de forma macro

O dinheiro só mora em dois lugares: o saldo da conta corrente e a posição investida. Cada seta do desenho abaixo é **uma** transação do PostgreSQL. Ela grava o pedido (`transaction` ou `investment_order`), os lançamentos do extrato (`ledger_entry`) e o novo saldo ou posição, e o `commit` é a última coisa que acontece. Antes de decidir, a API trava com `SELECT ... FOR UPDATE` toda linha que leu, sempre na mesma ordem. Ela também confere a `Idempotency-Key` já com a trava segurada, e um `UNIQUE` no banco serve de rede de segurança. Os preços dos produtos chegam uma vez por dia, no fechamento do dia, vindos das fontes externas (simuladas por um MockServer). Eles ficam gravados, então nenhum pedido de cliente depende de um serviço de fora estar no ar. O "hoje" do banco é uma data de movimento própria, que só avança pelo fechamento. É ela que permite mostrar "aplica no dia *t*, resgata no dia *t* + *n*" em segundos.





Considerado e descartado:

- **Saldo só como soma do extrato** (sem coluna `balance`): descartada porque toda leitura de saldo vira uma soma sobre o histórico inteiro, e a corrida entre dois pedidos continua existindo. Ganharia se o volume fosse pequeno e quiséssemos zero redundância. Ficamos com `balance` + `balance_after` em cada lançamento, e um teste confere que os dois batem.
- **`UPDATE ... SET balance = balance - x WHERE balance >= x`, sem trava explícita:** descartada porque a transferência precisa ler duas contas e o status delas antes de decidir. A trava explícita é mais fácil de explicar e de testar. Ganharia se só uma conta fosse afetada e o desempenho pesasse mais.
- **Uma conta de receitas do banco com saldo, creditada a cada tarifa:** descartada porque toda transferência travaria a *mesma* linha, e o banco inteiro faria uma transferência por vez. Ganharia com volume pequeno. A receita é a soma dos lançamentos `FEE`.
- **Buscar o preço lá fora na hora da aplicação:** descartada porque cada pedido do cliente passaria a depender de outro serviço, e dois clientes no mesmo dia poderiam ver preços diferentes. Ganharia com ativos de preço minuto a minuto, como ações.
- **Relógio real como "hoje":** descartada porque o dia seguinte só chegaria depois de 24 h, e não daria para testar nem demonstrar rendimento e vencimento. Ganharia sem rentabilidade para mostrar. Idade mínima e validade do perfil continuam no relógio real, porque são fatos regulatórios.

Implementação

Rotas

Toda rota exige o cabeçalho `INTERNAL-TOKEN`, menos `/` e `/health_check`. Sem ele, a resposta é `403`. Um recurso de outro cliente responde `404`, igual a um inexistente. As rotas marcadas como **idempotentes** exigem o cabeçalho `Idempotency-Key` (UUID); sem ele, `400` (`PPP000003`). Se a mesma chave vier com o mesmo corpo, a resposta é `200` com o corpo original e nada se move de novo. Se vier com um corpo diferente, `409` (`PPP000004`). Quem garante isso é a dupla `idempotency_key` + `request_hash` (SHA-256 do corpo) e o `UNIQUE` indicado. `ck` = `customer_key`, `ak` = `account_key`.

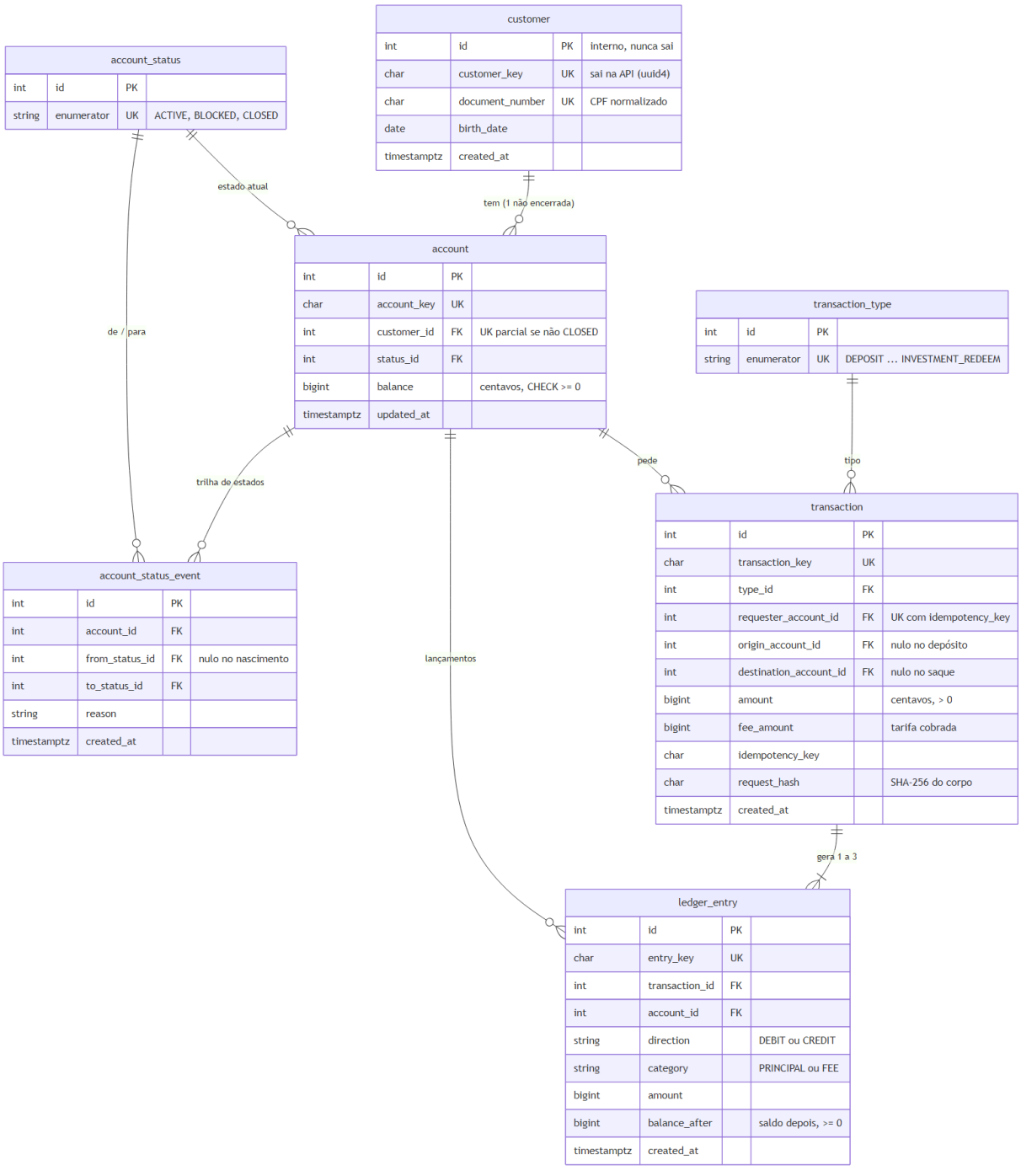
Método	Caminho	O que faz	Entrada (campos que importam)	Saídas (status e quando)
`POST`	`/customer`	Cadastra cliente	`name`, `document_number` (com ou sem pontuação), `birth_date`	`201` `customer_key`; `400` schema; `409` CPF já cadastrado; `422` CPF inválido ou menor de 18
`GET`	`/customer/{ck}`	Lê o cliente (CPF mascarado)	—	`200`; `404`
`POST`	`/customer/{ck}/account`	Abre a conta corrente (nasce `ACTIVE`, saldo 0)	—	`201` `account_key`; `404` cliente; `409` já tem conta não encerrada
`GET`	`/customer/{ck}/account/{ak}`	Saldo e status	—	`200`; `404`
`PATCH`	`/customer/{ck}/account/{ak}`	Bloqueia, desbloqueia ou encerra, gerando evento	`status`, `reason`	`200`; `400` status fora do enum; `404`; `409` transição proibida ou conta não vazia; `422` sem motivo
`GET`	`/customer/{ck}/account/{ak}/status_events`	Trilha de status	—	`200`; `404`

Método	Caminho	O que faz	Entrada (campos que importam)	Saídas (status e quando)
`POST`	`/customer/{ck}/account/{ak}/transaction`	Depósito, saque ou transferência (tarifa de R$ 2,00). **Idempotente**: `UNIQUE(requester_account_id, idempotency_key)`	`type`, `amount` (centavos), `destination_account_key`	`201` `transaction_key` e saldo; `200` repetição; `400` schema ou sem chave; `404` conta ou destino; `409` conta/destino não ativos, saldo insuficiente, chave reutilizada; `422` valor ≤ 0, acima do limite, origem = destino
`GET`	`/customer/{ck}/account/{ak}/transactions`	Extrato paginado, do mais novo ao mais antigo	`page`, `limit`, `from`, `to`, `type`	`200` `{items, page, limit, total}`; `400` filtro inválido; `404`
`GET`	`/customer/{ck}/account/{ak}/transaction/{tk}`	Comprovante	—	`200`; `404` (inexistente ou não envolve a conta)
`GET`	`/suitability_questions`	Perguntas e opções, sem os pontos	—	`200`
`POST`	`/customer/{ck}/suitability`	Responde o questionário e gera uma avaliação nova	`answers[{question_code, option_code}]`	`201` perfil, pontos, validade; `400` schema; `404`; `422` resposta faltando, repetida ou opção inválida
`GET`	`/customer/{ck}/suitability`	Perfil vigente (a avaliação mais recente)	—	`200`; `404` cliente ou nunca respondeu
`GET`	`/customer/{ck}/suitabilities`	Histórico de avaliações	—	`200`; `404`
`GET`	`/products`	Catálogo	`type`, `max_risk`	`200`
`GET`	`/product/{pk}`	Detalhe e preço do dia	—	`200`; `404`
`GET`	`/product/{pk}/prices`	Histórico de preços	`from`, `to`	`200`; `404`
`GET`	`/customer/{ck}/portfolio_suggestion`	Divide um valor pela carteira-modelo do perfil (não grava nada)	`amount`	`200` itens que somam exatamente `amount`; `400`; `404`; `409` sem perfil ou vencido; `422` abaixo de R$ 1.000
`POST`	`/customer/{ck}/investment_order`	Aplica ou resgata; o dinheiro sai ou entra na conta corrente. **Idempotente**: `UNIQUE(customer_id, idempotency_key)`	`product_key`, `side`, `amount` ou `redeem_all`	`201` `order_key`, quantidade, preço (resgate: custo, resultado, rentabilidade); `200` repetição; `400` schema ou sem chave; `404` produto; `409` sem conta ativa, sem perfil válido, saldo ou posição insuficiente, chave reutilizada; `422` abaixo do mínimo, fora do perfil, resgate antes do vencimento
`GET`	`/customer/{ck}/investment_orders`	Histórico de ordens	`page`, `limit`, `product_key`	`200` (produto inexistente: lista vazia); `400`; `404`
`GET`	`/customer/{ck}/investment_order/{ok}`	Detalhe da ordem	—	`200`; `404`
`GET`	`/customer/{ck}/positions`	Posição, valor do dia e rentabilidade	`include_closed`	`200`; `400`; `404`

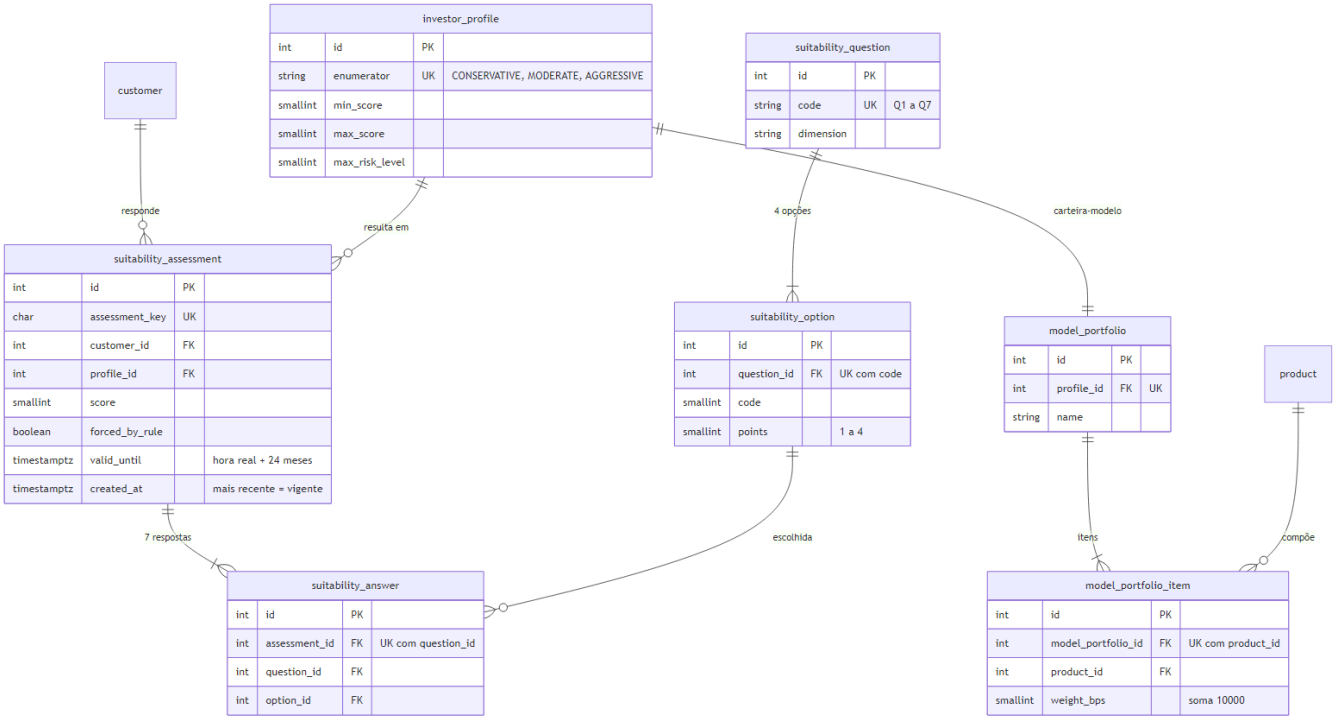
Método	Caminho	O que faz	Entrada (campos que importam)	Saídas (status e quando)
`GET`	`/internal/business_date`	Data de movimento atual	—	`200`
`POST`	`/internal/day_close`	Fecha o dia: busca preços e taxas lá fora, liquida o que venceu e abre o próximo dia útil. Clique duplo não fecha dois dias (trava + data conferida)	`close_date`	`201` nova data, preços, taxas; `400`; `409` `close_date` não é a data atual; `502` fonte fora do ar ou resposta sem sentido (nada gravado)
`GET`	`/internal/revenue`	Receita realizada (tarifas) e estimada (fundo, spread, float)	`from`, `to`	`200`; `400` período invertido ou maior que 366 dias

Banco de Dados (Somente diagrama)

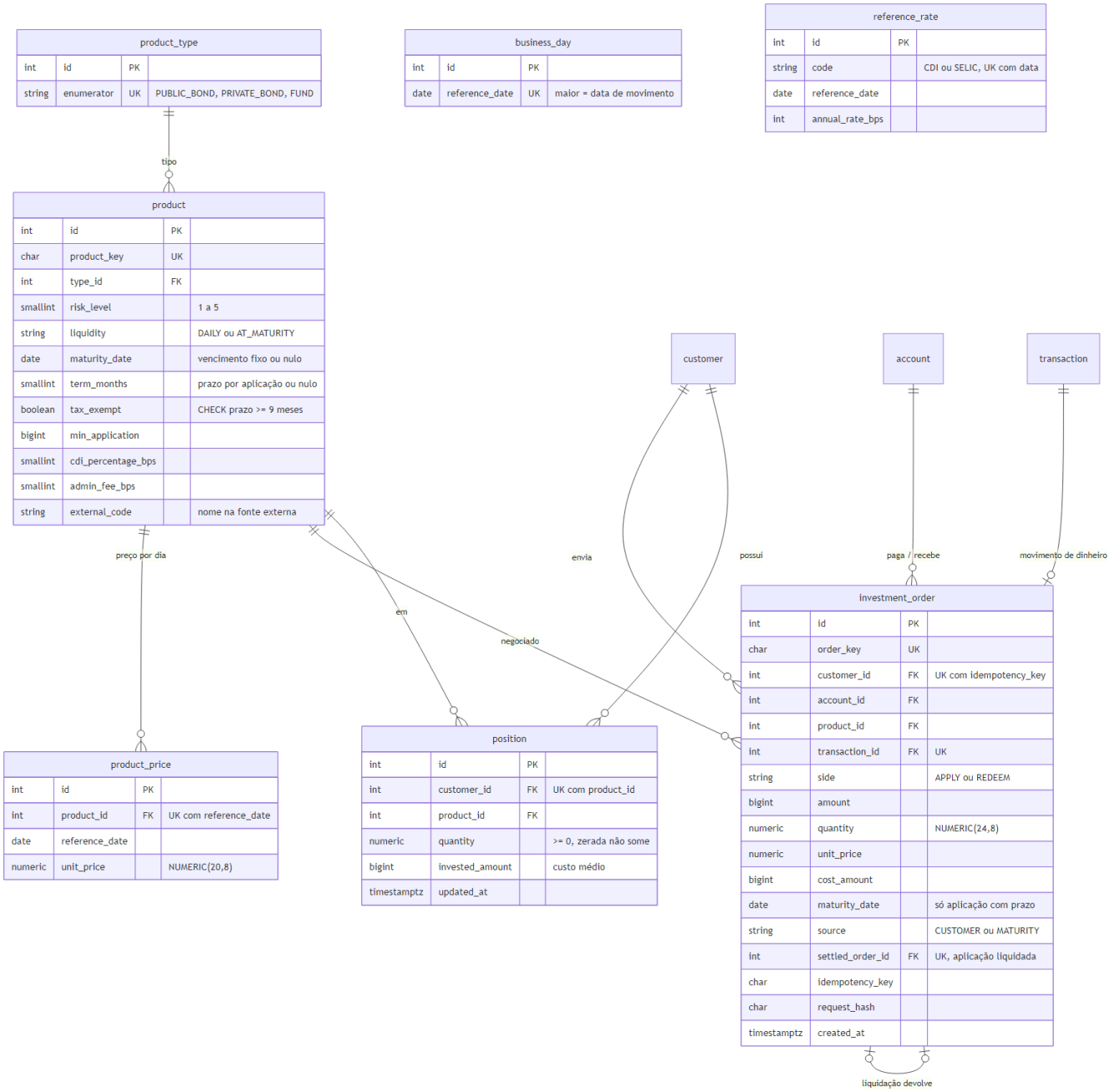
Núcleo bancário



Perfil do investidor e carteira sugerida



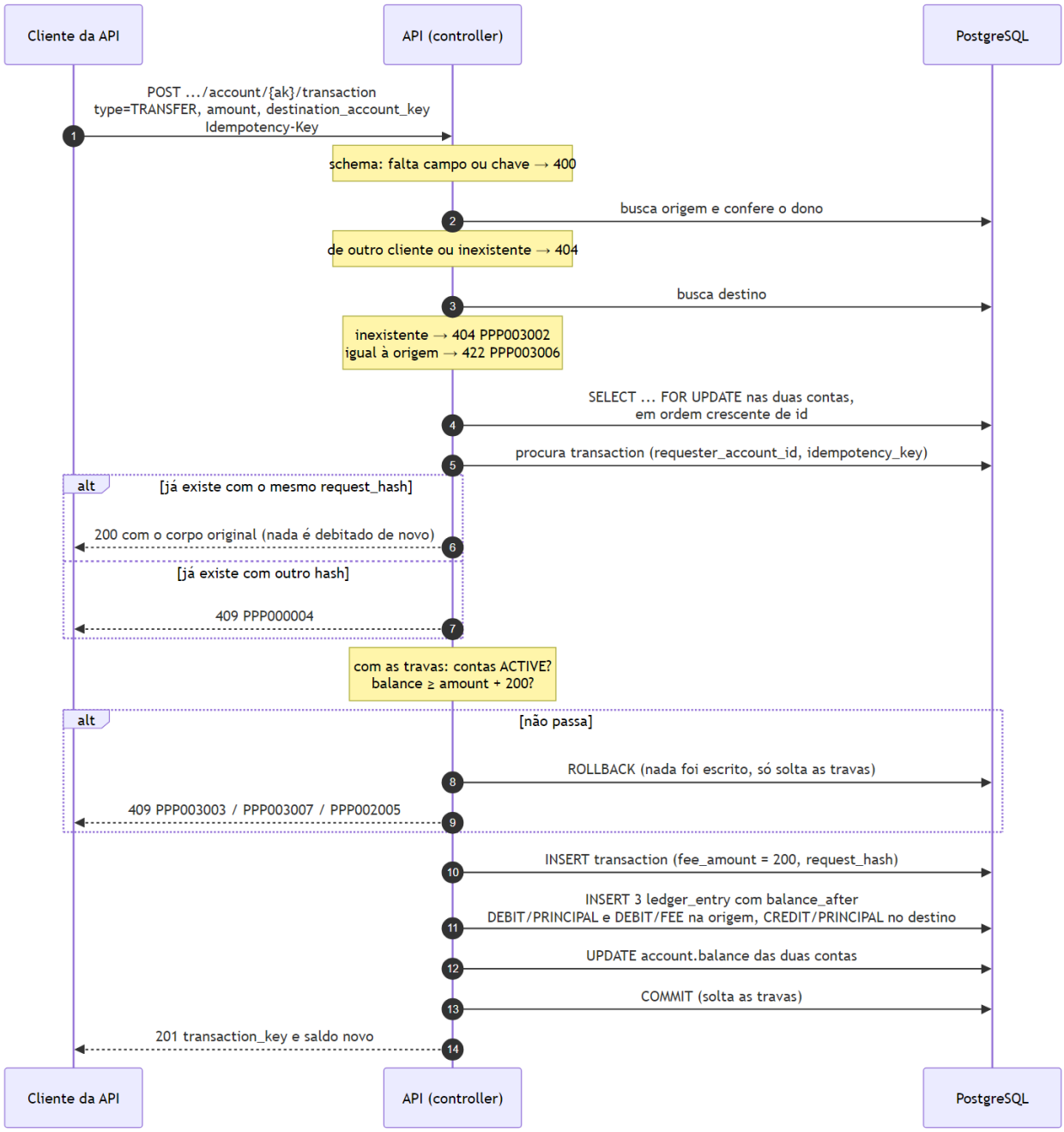
Produtos, preços, ordens e posição



Fluxos

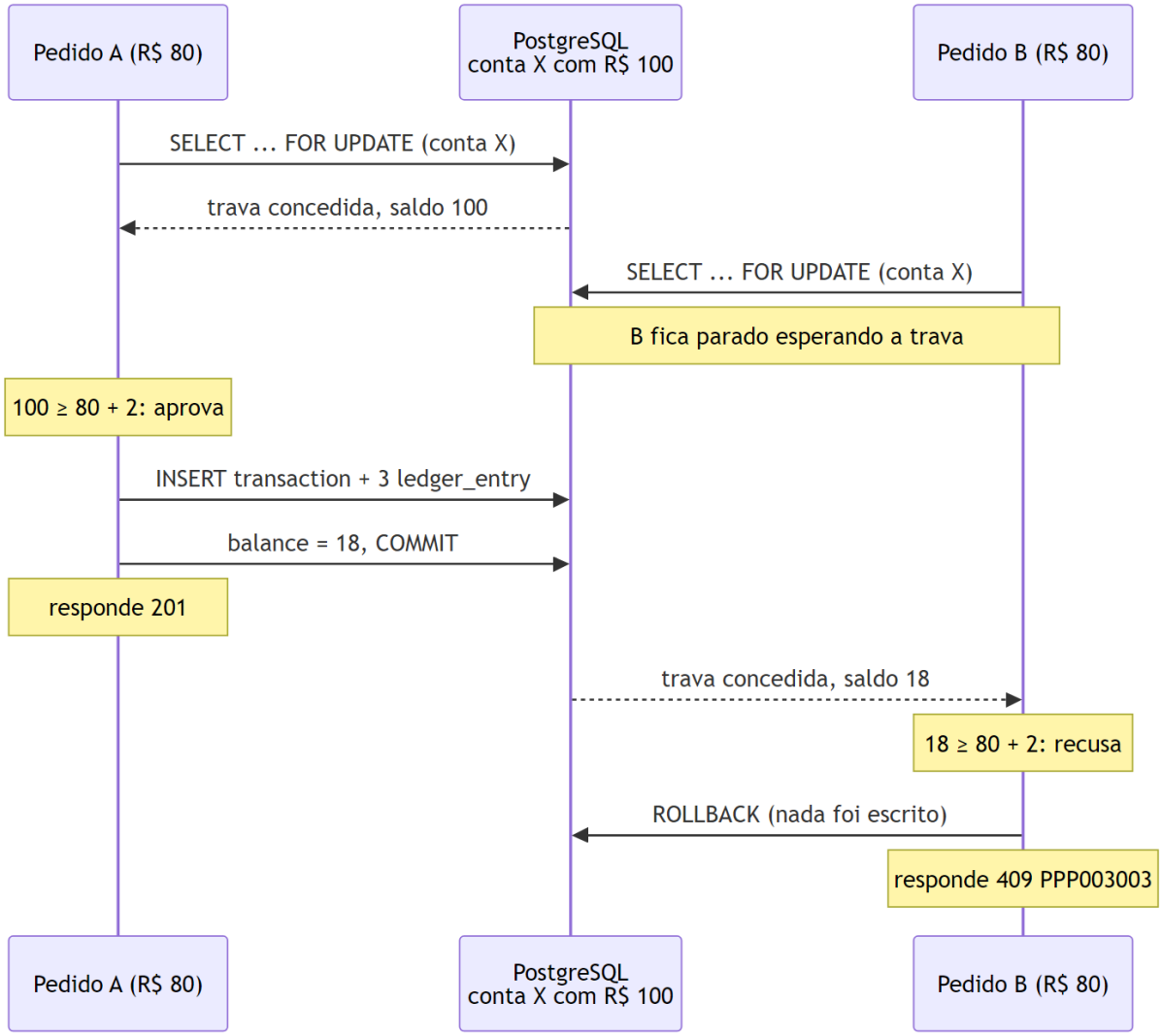
Toda operação que move dinheiro segue a mesma ordem: **ler → travar → checar idempotência → decidir → escrever →** `commit`. Nenhuma escrita acontece antes da última verificação, então uma recusa não tem nada a desfazer. E como o `commit` é um só, um imprevisto no meio das escritas não deixa nada pela metade: o middleware de sessão dá `rollback` e o banco volta a como estava.

Transferência — caminho feliz



Transferência — falha: saldo insuficiente (a mais provável). O passo de decisão recusa com as duas contas travadas. Como nada foi escrito, o `rollback` só solta as travas. O cliente recebe `409` (`PPP003003`) e os dois saldos continuam iguais.

Transferência — falha: duas transferências da mesma conta ao mesmo tempo (a mais cara). A conta tem R$ 100 e chegam dois pedidos de R$ 80.

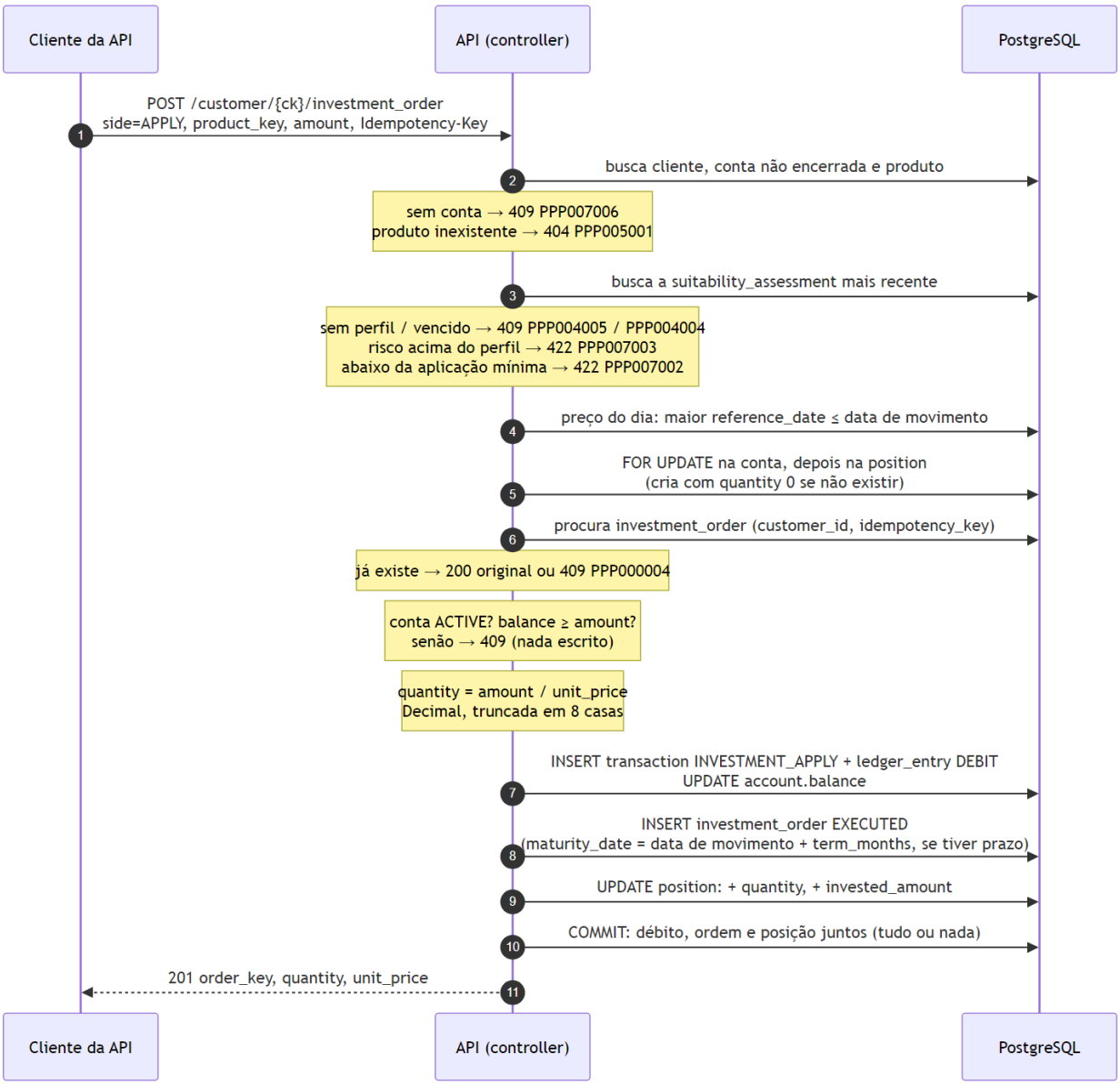


Sem a trava, A e B leriam R$ 100, os dois aprovariam e R$ 160 sairiam de uma conta com R$ 100 (*lost update*). O `CHECK (balance >= 0)` ainda barraria o segundo, mas com um `500`, que é um ramo que ninguém desenhou.

Transferência — falha: o cliente reenvia o mesmo pedido. Se o original ainda está rodando, o reenvio espera na trava. Depois ele encontra a `transaction` já gravada e devolve o corpo original (`200`). A checagem fica *depois* da trava de propósito: se viesse antes, o reenvio passaria por ela, esperaria a trava e receberia um falso "saldo insuficiente".

Transferência — falha: A → B e B → A ao mesmo tempo. Se cada pedido travasse primeiro a própria origem, cada um seguraria uma conta esperando a outra (*deadlock*). Travando sempre em ordem crescente de `id`, os dois disputam a mesma primeira trava: um passa e o outro espera.

Aplicação — caminho feliz



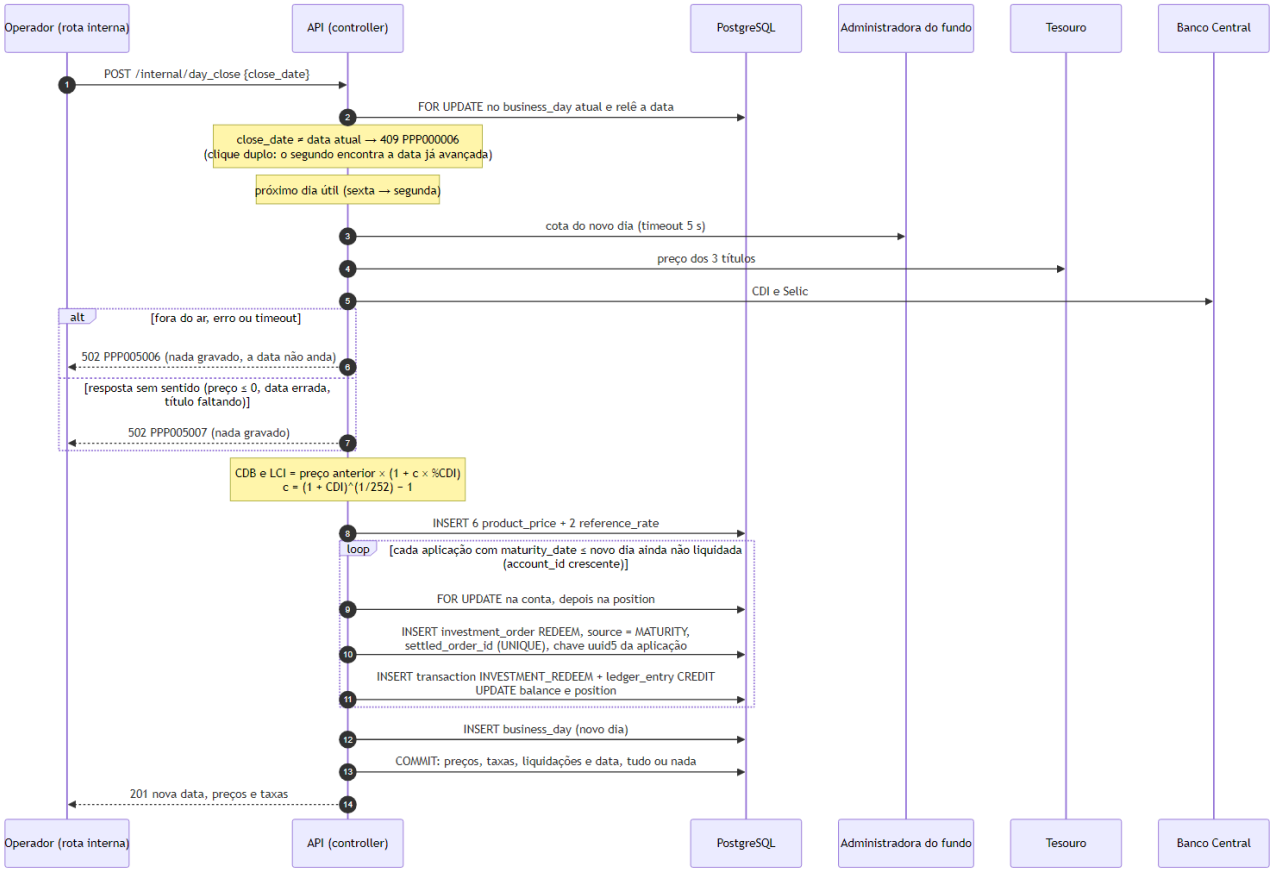
A ordem das travas é sempre *conta, depois posição*. Como cada cliente tem uma só conta aberta, a trava da conta enfileira todas as ordens daquele cliente, e a posição pode ser criada sem que duas ordens tentem criá-la juntas.

Aplicação — falha: produto acima do perfil (a mais provável). A recusa acontece antes de qualquer trava ou escrita, com `422` (`PPP007003`). Perfil vencido ou inexistente recusa no mesmo ponto, com `409`.

Aplicação — falha: exceção depois do débito (a mais cara). O débito já está na sessão, mas a ordem ou a posição não. O `rollback` desfaz débito, lançamento e ordem juntos: dinheiro nunca sai da conta sem virar posição.

Resgate — o que muda. Não há checagem de perfil: o cliente sempre pode tirar o dinheiro. Com a posição travada, a API confere `quantity` ≥ quantidade a resgatar (senão `409` `PPP007004`). Produto `AT_MATURITY` antes do vencimento da aplicação dá `422` (`PPP007005`). O crédito é um `INVESTMENT_REDEEM`, e a posição perde a quantidade e o custo na mesma proporção. Quando zera, a posição fica com `quantity = 0` e não é apagada.

Fechamento do dia — caminho feliz



Fechamento — falha: serviço externo fora do ar (a mais provável). Erro, timeout de 5 s ou conexão derrubada dão `502` (`PPP005006`). Nada foi escrito: a data não anda e vale o último preço publicado. Basta chamar de novo. Uma resposta sem sentido (cota = 0, título faltando) dá `502` (`PPP005007`), pelo mesmo caminho. Gravar um preço errado seria pior, porque preço publicado não se reescreve.

Fechamento — falha: erro no meio da liquidação (a mais cara). Preços gravados e uma aplicação já paga, mas a próxima falha. Como tudo está num `commit` só, o `rollback` desfaz preços, taxas e créditos juntos, e ninguém recebe pela metade. Uma aplicação nunca é paga duas vezes: depois de paga, existe a ordem que aponta para ela, e o `UNIQUE(settled_order_id)` barra a segunda.

Principal desafio

- **Qual é:** mover dinheiro entre contas, e entre conta e investimento, sem criar, duplicar ou sumir com um centavo.
- **Por que é difícil:** cada operação faz de três a cinco escritas em tabelas diferentes, e a solução óbvia (ler o saldo, checar, escrever) quebra em três cenários. Com dois pedidos simultâneos, os dois leem o mesmo saldo e R$ 160 saem de R$ 100. Com o cliente reenviando depois de um timeout, a transferência sai duas vezes. Com uma falha no meio, o débito fica gravado sem a posição.
- **Como o desenho resolve:**
  - uma transação de banco por operação, com `commit` só na última linha: tudo ou nada;
  - `SELECT ... FOR UPDATE` em toda linha lida para decidir, sempre na mesma ordem (contas por `id` crescente, depois posição), o que também evita *deadlock*;
  - idempotência conferida *com a trava segurada*, com o `UNIQUE(..., idempotency_key)` como rede de segurança;
  - o invariante `account.balance` = Σ créditos − Σ débitos de `ledger_entry`: o `balance_after` de cada lançamento guarda o saldo naquele instante, e um teste black box confere que o extrato reconstrói o saldo.